

“Adversarial Examples” for Proof-of-Learning

Rui Zhang[†], Jian Liu^{†*}, Yuan Ding, Zhibo Wang, Qingbiao Wu, and Kui Ren

Zhejiang University

Email: {zhangrui98, liujian2411, dylant, zhibowang, qbwu, kuiren}@zju.edu.cn

Abstract—In S&P ’21, Jia et al. proposed a new concept/mechanism named proof-of-learning (PoL), which allows a prover to demonstrate ownership of a machine learning model by proving integrity of the training procedure. It guarantees that an adversary cannot construct a valid proof with less cost (in both computation and storage) than that made by the prover in generating the proof.

A PoL proof includes a set of intermediate models recorded during training, together with the corresponding data points used to obtain each recorded model. Jia et al. claimed that an adversary merely knowing the final model and training dataset cannot efficiently find a set of intermediate models with correct data points.

In this paper, however, we show that PoL is vulnerable to “adversarial examples”! Specifically, in a similar way as optimizing an adversarial example, we could make an arbitrarily-chosen data point “generate” a given model, hence efficiently generating intermediate models with correct data points. We demonstrate, both theoretically and empirically, that we are able to generate a valid proof with significantly less cost than generating a proof by the prover.

I. INTRODUCTION

Recently, Jia et al. [12] propose a concept/mechanism named *proof-of-learning* (PoL), which allows a prover $\mathcal{T}$ to prove that it has performed a specific set of computations to train a machine learning model; and a verifier $\mathcal{V}$ can verify correctness of the proof with significantly less cost than training the model. This mechanism can be applied in at least two settings. First, when the intellectual property of a model owner is infringed upon (e.g., by a model stealing attack [18], [24], [25]), it allows the owner to claim ownership of the model and resolve the dispute. Second, in the setting of federated learning [17], where a model owner distributes the training process across multiple workers, it allows the model owner to verify the integrity of the computation performed by these workers. This could prevent Byzantine workers from conducting denial-of-service attacks [4].

PoL mechanism. In their proposed mechanism [12], $\mathcal{T}$ provides a PoL proof that includes: (i) the training dataset, (ii) the intermediate model weights at periodic intervals during training $W_0, W_k, W_{2k}, \dots, W_T$, and (iii) the corresponding indices of the data points used to train each intermediate model. With a PoL proof, one can replicate the path all the way from the initial model weights W_0 to the final model weights W_T to be fully confident that $\mathcal{T}$ has indeed performed the computation required to obtain the final model.

During verification, $\mathcal{V}$ first verifies the provenance of the initial model weights W_0 : whether it is sampled from the required initialization distribution; and then recomputes a subset of the intermediate models to confirm the validity of the sequence provided. However, $\mathcal{V}$ may not be able to reproduce the same sequence due to the noise arising from the hardware and low-level libraries. To this end, they allow a distance between the recomputed model and its corresponding model in PoL. Namely, for any W_t , $\mathcal{V}$ performs a series of k updates to arrive at W'_{t+k} , which is compared to the purported W_{t+k} . They tolerate:

$$d(W_{t+k}, W'_{t+k}) \leq \delta,$$

where d represents a distance that could be l_1 , l_2 , l_∞ or $\cos$, and δ is the verification threshold that should be calibrated before verification starts.

Jia et al. [12] claimed in their paper that an adversary $\mathcal{A}$ can never construct a valid a PoL with less cost (in both computation and storage) than that made by $\mathcal{T}$ in generating the proof (a.k.a. *spoof a PoL*). However, they did not provide a proof to back their claim.

Our contribution. By leveraging the idea of generating adversarial examples, we successfully spoof a PoL!

In the PoL threat model, Jia et al. [12] assumed that “ $\mathcal{A}$ has full access to the training dataset, and can modify it”. Thanks to this assumption, we can slightly modify a data point so that it can update a model and make the result pass the verification. In more detail, given the training dataset and the final model weights W_T , $\mathcal{A}$ randomly samples all intermediate model weights in a PoL: $W_0, W_k, W_{2k} \dots$ (only W_0 needs to be sampled from the given distribution). For any two neighboring model weights (W_{t-k}, W_t), $\mathcal{A}$ picks batches of data points $(\mathbf{X}, \mathbf{y})$ from D , and keeps manipulating $\mathbf{X}$ until:

$$d(\text{update}(W_{t-k}, (\mathbf{X}, \mathbf{y})), W_t) \leq \delta.$$

The mechanism for generating adversarial examples ensures that the noise added to $\mathbf{X}$ is minimized.

We further optimize our attack by sampling $W_0, W_k, W_{2k} \dots$ in a way such that:

$$d(W_t, W_{t-k}) < \delta, \forall 0 < t < T \text{ and } t \bmod k = 0.$$

With this condition, it becomes much easier for the “adversarial” $\mathbf{X}$ to converge, hence making our attack more efficient.

We empirically evaluate our attacks in both reproducibility and spoof cost. We reproduced the results in [12] as baselines for our evaluations. Our experimental results show that,

[†]Rui Zhang and Jian Liu are co-first authors.

*Jian Liu is the corresponding author.

in most cases of our setting, our attacks introduce smaller reproduction errors and less cost than the baselines.

Organization. In the remainder of this paper, we first provide a brief introduction to PoL in Section II. Then, we formally describe our attack in Section III and extensively evaluate it in Section IV. In Section V, we provide some countermeasures. Section VI compares our attacks to closely related work.

Notations. We introduce new notations as needed. A summary of frequent notations appears in Table I.

Table I
SUMMARY OF NOTATIONS

Notation	Description
$\mathcal{T}$	prover
$\mathcal{V}$	verifier
$\mathcal{A}$	attacker
D	dataset
f_W	machine learning model
W	model weights
E	number of epochs
S	number of steps per epoch
T	number of steps in $\mathcal{P}(\mathcal{T}, f_{W_T})$ $T = E \cdot S$
T'	number of steps in $\mathcal{P}(\mathcal{A}, f_{W_T})$
Q	number of models verified per epoch
N	number of steps in generating an “adversarial example”
k	number of batches in a checkpointing interval
$d()$	distance that could be l_1, l_2, l_∞ or $\cos$
δ	verification threshold
γ	$\gamma \ll \delta$
ζ	distribution for W_0
η	learning rate
ε	reproduction error
$\mathbf{X}$	batch of data points
$\mathbf{y}$	batch of labels
$\mathbf{R}$	batch of noise

II. PROOF-OF-LEARNING

In this section, we provide a brief introduction to proof-of-learning (PoL). We refer to [12] for more details

A. PoL definition

PoL allows a prover $\mathcal{T}$ to demonstrate ownership of a machine learning model by proving the integrity of the training procedure. Namely, during training, $\mathcal{T}$ accumulates some secret information associated with training, which is used to construct the PoL proof $\mathcal{P}(\mathcal{T}, f_{W_T})$. When the integrity of the computation (or model ownership) is under debate, an honest and trusted verifier $\mathcal{V}$ validates $\mathcal{P}(\mathcal{T}, f_{W_T})$ by querying $\mathcal{T}$ for a subset (or all of) the secret information, under which $\mathcal{V}$

should be able to ascertain if the PoL is valid or not. A PoL proof is formally defined as follows:

Definition 1. A PoL proof generated by a prover $\mathcal{T}$ is defined as $\mathcal{P}(\mathcal{T}, f_{W_T}) = (\mathbb{W}, \mathbb{I}, \mathbb{H}, \mathbb{A})$, where (a) $\mathbb{W}$ is a set of intermediate model weights recorded during training, (b) $\mathbb{I}$ is a set of information about the specific data points used to train each intermediate model, (c) $\mathbb{H}$ is a set of signatures generated from these data points, and (d) $\mathbb{A}$ incorporates auxiliary information training the model such as hyperparameters, model architecture, optimizer and loss choices¹.

An adversary $\mathcal{A}$ might wish to spoof $\mathcal{P}(\mathcal{T}, f_{W_T})$ by spending less computation and storage than that made by $\mathcal{T}$ in generating the proof. By spoofing, $\mathcal{A}$ can claim that it has performed the computation required to train f_{W_T} . A PoL mechanism should guarantee:

- The cost of verifying the PoL proof by $\mathcal{V}$ should be smaller than the cost (in both computation and storage) of generating the proof by $\mathcal{T}$.
- The cost of any spoofing strategy attempted by any $\mathcal{A}$ should be larger than the cost of generating the proof.

B. Threat Model

In [12], any of the following cases is considered to be a successful spoof by $\mathcal{A}$:

- 1) *Retraining-based spoofing*: $\mathcal{A}$ produced a PoL for f_{W_T} that is exactly the same as the one produced by $\mathcal{T}$, i.e., $\mathcal{P}(\mathcal{A}, f_{W_T}) = \mathcal{P}(\mathcal{T}, f_{W_T})$.
- 2) *Stochastic spoofing*: $\mathcal{A}$ produced a valid PoL for f_{W_T} , but it is different from the one produced by $\mathcal{T}$ i.e., $\mathcal{P}(\mathcal{A}, f_{W_T}) \neq \mathcal{P}(\mathcal{T}, f_{W_T})$.
- 3) *Structurally Correct Spoofing*: $\mathcal{A}$ produced an invalid PoL for f_{W_T} but it can pass the verification.
- 4) *Distillation-based Spoofing*: $\mathcal{A}$ produced a valid PoL for an approximated model, which has the same run-time performance as f_{W_T} .

The following adversarial capabilities are assumed in [12]:

- 1) $\mathcal{A}$ has full knowledge of the model architecture, model weights, loss function and other hyperparameters.
- 2) $\mathcal{A}$ has full access to the training dataset D and can modify it. **This assumption is essential to our attacks.**
- 3) $\mathcal{A}$ has no knowledge of $\mathcal{T}$ ’s strategies about batching, parameter initialization, random generation and so on.

C. PoL Creation

The PoL creation process is shown in Algorithm 1, which is taken from [12] and slightly simplified by us. $\mathcal{T}$ first initializes the weights W_0 according to an initialization strategy $\text{init}(\zeta)$ (line 2), where ζ is the distribution to draw the weights from. If the initial model is obtained from elsewhere, a PoL is required for the initial model itself as well. We omit this detail in our paper for simplicity.

¹For simplicity, we omit $\mathbb{A}$ in this paper and denote a PoL proof as $\mathcal{P}(\mathcal{T}, f_{W_T}) = (\mathbb{W}, \mathbb{I}, \mathbb{H})$.

Algorithm 1: PoL Creation (taken from [12])

Input: D, k, E, S, ζ
Output: PoL proof: $\mathcal{P}(\mathcal{T}, f_{W_T}) = (\mathbb{W}, \mathbb{I}, \mathbb{H})$

```

1  $\mathbb{W} \leftarrow \{\} \quad \mathbb{I} \leftarrow \{\} \quad \mathbb{H} \leftarrow \{\}$ 
2  $W_0 \leftarrow \text{init}(\zeta)$  initialize  $W_0$ 
3 for  $e = 0 \rightarrow E - 1$  do
4    $I \leftarrow \text{getBatches}(D, S)$ 
5   for  $s = 0 \rightarrow S - 1$  do
6      $t := e \cdot S + s$ 
7      $W_{t+1} \leftarrow \text{update}(W_t, D[I[s]])$ 
8      $\mathbb{I}.\text{append}(I[s])$ 
9      $\mathbb{H}.\text{append}(h(D[I[s]]))$   $h()$  is for
computing the signature
10    if  $t \bmod k = 0$  then
11       $\mathbb{W}.\text{append}(W_t)$ 
12    else
13       $\mathbb{W}.\text{append}(\text{nil})$ 
14    end
15  end
16 end
```

For each epoch, $\mathcal{T}$ gets S batches of data points from the dataset D via $\text{getBatches}(D, S)$ (Line 4), the output of which is a list of S sets of data indices. In each step s of the epoch e , the model weights are updated with a batch of data points in D indexed by $I[s]$ (Line 7). The update function leverages a suitable optimizer implementing a variant of gradient descent. $\mathcal{T}$ records the updated model W_t for every k steps (Line 11), hence k is a parameter called checkpointing interval and $\frac{1}{k}$ is then the checkpointing frequency. To ensure that the PoL proof will be verified with the same data points as it was trained on, $\mathcal{T}$ includes a signature of the training data (Line 9) along with the data indices (Line 8).

D. PoL Verification

Algorithm 2 shows the PoL verification process. $\mathcal{V}$ first checks if W_0 was sampled from the required distribution using a statistical test (Line 1). Once every epoch, $\mathcal{V}$ records the distances between each two neighboring models in mag (line 7-9); sort mag to find Q largest distances and verify the corresponding models and data samples via verifyEpoch (Line 12-13). Notice that there are at most $\lfloor \frac{S}{k} \rfloor$ distances in each epoch, hence $Q \leq \lfloor \frac{S}{k} \rfloor$.

In the verifyEpoch function, $\mathcal{V}$ first loads the batch of indexes corresponding to the data points used to update the model from W_t to W_{t+k} . Then, it attempts to reproduce W_{t+k} by performing a series of k updates to arrive at W'_{t+k} . Notice that $W'_{t+k} \neq W_{t+k}$ due to the noise arising from the hardware and low-level libraries such as cuDNN [7]. The reproduction error for the t -th model is defined as:

$$\varepsilon_{repr}(t) = d(W_{t+k}, W'_{t+k}),$$

where d represents a distance that could be l_1 , l_2 , l_∞ or $\cos$. It is required that:

Algorithm 2: PoL Verification (taken from [12])

Input: $\mathcal{P}(\mathcal{T}, f_{W_T}), D, k, E, S, \zeta$
Output: success / fail

```

1 if  $\text{verifyInitialization}(\mathbb{W}[0]) = \text{fail}$  then
2   return fail
3 end
4  $e \leftarrow 0$ 
5  $\text{mag} \leftarrow \{\}$ 
6 for  $t = 0 \rightarrow T - 1$  do
7   if  $t \bmod k = 0 \wedge t \neq 0$  then
8      $\text{mag}.\text{append}(d(\mathbb{W}[t], \mathbb{W}[t - k]))$ 
9   end
10   $e_t = \lfloor \frac{t}{S} \rfloor$ 
11  if  $e_t = e + 1$  then
12     $\text{idx} \leftarrow \text{sortedIndices}(\text{mag}, \downarrow)$ 
13    if  $\text{verifyEpoch}(\text{idx}) = \text{fail}$  then
14      return fail
15    end
16     $e \leftarrow e_t, \text{mag} \leftarrow \{\}$ 
17  end
18 end
19 return success
20
21 function  $\text{verifyEpoch}(\text{idx})$ 
22   for  $q = 1 \rightarrow Q$  do
23      $t := \text{idx}[q - 1]$ 
24      $\text{verifyDataSignature}(\mathbb{H}[t], D[\mathbb{I}[t]])$ 
25      $W'_t \leftarrow \mathbb{W}[t]$ 
26     for  $i = 0 \rightarrow (k - 1)$  do
27        $I_{t+i} \leftarrow \mathbb{I}[t + i]$ 
28        $W'_{t+i+1} \leftarrow \text{update}(W'_{t+i}, D[\mathbb{I}[t + i]])$ 
29     end
30     if  $d(W'_{t+k}, \mathbb{W}[t + k]) > \delta$  then
31       return fail
32     end
33   end
34 end
```

$$\max_t(\varepsilon_{repr}(t)) \ll d_{ref},$$

where $d_{ref} = d(W_T^1, W_T^2)$ is the distance between two models W_T^1 and W_T^2 trained with the same architecture, dataset, and initialization strategy, but with different batching strategies and potentially different initial model weights. A *verification threshold* δ that satisfies:

$$\max_t(\varepsilon_{repr}(t)) < \delta < d_{ref},$$

should be calibrated before verification starts. In their experiments, Jia et al. [12] adopted a normalized reproduction error:

$$||\varepsilon_{repr}(t)|| = \frac{\max_t(\varepsilon_{repr}(t))}{d_{ref}}$$

to evaluate the reproducibility.

In the end, we remark that the number of steps T in PoL verification (Algorithm 2) could be different from that in PoL

creation (Algorithm 1), because $\mathcal{A}$ could come up with either a stochastic spoofing or a structurally correct spoofing.

III. ATTACK METHODOLOGY

In this section, we describe our attacks in detail. All of our attacks are stochastic spoofing: the PoL proof generated by the adversary $\mathcal{A}$ is not exactly the same as the one provided by the prover $\mathcal{T}$ (in particular, with a smaller number of steps T'), but can pass the verification.

Figure 1 shows the basic idea of our attacks: the adversary $\mathcal{A}$ first generates dummy model weights: $W_0, \dots, W_{T-1}$ (serving as $\mathbb{W}$); and then generates “adversarial examples” (serving as $\mathbb{I}$) for each pair of neighboring models. An adversarial example is an instance added with small and intentional perturbations so that a machine learning model will make a false prediction on it. In a similar way as optimizing an adversarial example, we could make an arbitrarily-chosen date point “generate” a given model (we call it *adversarial optimization*), hence making $(\mathbb{W}, \mathbb{I})$ pass the verification.

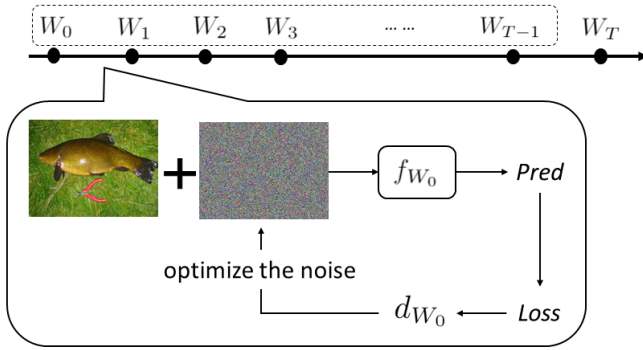


Figure 1. Basic idea of our attacks. The adversary first generates dummy model weights: $W_0, \dots, W_{T-1}$ (serving as $\mathbb{W}$); and then generates “adversarial examples” (serving as $\mathbb{I}$) for each pair of neighboring models.

Recall that one requirement for a spoof to succeed is that $\mathcal{A}$ should spend less cost than the PoL generation process described in Algorithm 1 (which is $T = E \cdot S$ times of update). Next, we show how we achieve this.

A. Attack I

Our first insight is that there is no need to construct an adversarial example for every pair of neighboring models. Instead, $\mathcal{A}$ could simply update the model from W_0 to W_{T-1} using original data points, and construct an “adversarial example” only from W_{T-1} to W_T . In this case, $\mathcal{A}$ only needs to construct a single “adversarial example” for the whole attacking process. Furthermore, $\mathcal{A}$ could use a smaller number of steps, denoted as T' .

Algorithm 3 shows our first attack. From W_0 to $W_{T'-1}$, it works in the same way as PoL creation (cf. Algorithm 1). For $W_{T'-1}$, the batch of inputs $(\mathbf{X}, \mathbf{y})$ must be manipulated s.t.:

$$d(W_{T'}, W_T) \leq \delta.$$

Line 22-28 show how $\mathcal{A}$ manipulates $\mathbf{X}$. Specifically, $\mathcal{A}$ first initializes a batch of noise $\mathbf{R}$ as zeros (line 23). Then, it feeds

Algorithm 3: Attack I

Input: $D, f_{W_T}, \delta, \zeta, k, E, S$
Output: PoL spoof: $\mathcal{P}(\mathcal{A}, f_{W_T}) = (\mathbb{W}, \mathbb{I}, \mathbb{H})$
 updated dataset: D

```

1  $\mathbb{W} \leftarrow \{\}$   $\mathbb{I} \leftarrow \{\}$   $\mathbb{H} \leftarrow \{\}$ 
2  $\mathbb{W}.\text{append}(\text{init}(\zeta))$  initialize and append  $W_0$ 
3 for  $t = 1 \rightarrow T'$  do  $T' \bmod k = 0$ 
4    $\mathbb{I}.\text{append}(\text{getBatch}(D))$ 
5   if  $t < T'$  then
6      $W_t \leftarrow \text{update}(W_{t-1}, D[\mathbb{I}[t-1]])$ 
7     if  $t \bmod k = 0$  then
8        $\mathbb{W}.\text{append}(W_t)$ 
9     else
10       $\mathbb{W}.\text{append}(\text{nil})$ 
11    end
12  else
13     $\text{updateDataPoints}(W_{t-1}, W_T)$ 
14  end
15   $\mathbb{H}.\text{append}(h(D[\mathbb{I}[t-1]]))$ 
16 end
17
18 function  $\text{updateDataPoints}(W_{t-1}, W_t)$ 
19    $W'_{t-1} := W_{t-1}$ 
20    $(\mathbf{X}, \mathbf{y}) \leftarrow D[\mathbb{I}[t-1]]$ 
21    $W'_t \leftarrow \text{update}(W'_{t-1}, (\mathbf{X}, \mathbf{y}))$ 
22   while  $d(W'_t, W_t) > \delta$  do
23      $\mathbf{R} \leftarrow \text{zeros}$ 
24      $\nabla_{W'_{t-1}} \leftarrow -\frac{\partial}{\partial W'_{t-1}} L(f_{W'_{t-1}}(\mathbf{X} + \mathbf{R}), \mathbf{y})$ 
25      $\mathbb{D}_{t-1} \leftarrow d(W'_{t-1} + \eta \nabla_{W'_{t-1}}, W_t) + d(\mathbf{R}, 0)$ 
26      $\mathbf{R} \leftarrow \mathbf{R} - \eta' \nabla_{\mathbf{R}} \mathbb{D}_{t-1}$ 
27      $W'_t \leftarrow \text{update}(W'_{t-1}, (\mathbf{X} + \mathbf{R}, \mathbf{y}))$ 
28   end
29    $D[\mathbb{I}[t-1]] := (\mathbf{X} + \mathbf{R}, \mathbf{y})$ 
30 end
```

$(\mathbf{X} + \mathbf{R})$ to $f_{W'_{t-1}}$ and gets the gradients $\nabla_{W'_{t-1}}$ (line 25). Next, $\mathcal{A}$ optimizes $\mathbf{R}$ by minimizing the following distance (line 26-27):

$$\mathbb{D}_{t-1} \leftarrow d(W'_{t-1} + \eta \nabla_{W'_{t-1}}, W_t) + d(\mathbf{R}, 0).$$

This distance needs to be differentiable so that $\mathbf{R}$ can be optimized using standard gradient-based methods². Notice that this optimization requires 2nd order derivatives. We assume that f_W is twice differentiable, which holds for most modern machine learning models and tasks.

Clearly, the PoL spoof $\mathcal{P}(\mathcal{A}, f_{W_T}) = (\mathbb{W}, \mathbb{I}, \mathbb{H})$ generated by Attack I can pass the verification process described in Algorithm 2. It requires T' times of `update` (Line 21) plus N times of adversarial optimization (Line 23-27) (where N is the times that the **while** loop runs). Recall that our focus is stochastic spoofing: the PoL proof generated by $\mathcal{A}$ is not

²Specificly, we use L-BFGS for adversarial optimization.

exactly the same as the one provided by $\mathcal{T}$, but can pass the verification. Therefore, we can use a T' that is much smaller than T . However, N could be large and sometimes even cannot converge. Next, we show how we optimize the attack so that a small N is able to make the adversarial optimization converge.

B. Attack II

The intuition for accelerating adversarial optimization is to sample the intermediate model weights in a way s.t.:

$$d(W_t, W_{t-k}) \leq \delta, \forall 0 < t < T \text{ and } t \bmod k = 0,$$

This brings at least three benefits:

- 1) “Adversarial examples” become easier to be optimized.
- 2) The k batches of “adversarial examples” in each check-pointing interval can be optimized together. (We defer to explain this benefit in Attack III.)
- 3) Intermediate model performance can be guaranteed. (Recall that $\mathcal{V}$ might check model performance periodically.)

Algorithm 4: Attack II

Input: $D, f_{W_T}, \delta, \gamma, \zeta, k, E, S$
Output: PoL spoof: $\mathcal{P}(\mathcal{A}, f_{W_T}) = (\mathbb{W}, \mathbb{I}, \mathbb{H})$
 updated dataset: D

```

1  $\mathbb{W} \leftarrow \{\}$   $\mathbb{I} \leftarrow \{\}$   $\mathbb{H} \leftarrow \{\}$ 
2  $\mathbb{W}.\text{append}(\text{init}W_0(\zeta, W_T))$            initialize and
    $\text{append } W_0$ 
3 for  $t = 1 \rightarrow T'$  do                      $T' \bmod k = 0$ 
4    $\mathbb{I}.\text{append}(\text{getBatch}(D))$ 
5   if  $t \bmod k = 0$  then
6     if  $t < T'$  then                     no need to append  $W_T$ 
7        $\text{sample } W_t \text{ s.t., } d(W_t, W_{t-k}) \leq \delta$ 
        $\mathbb{W}.\text{append}(W_t)$ 
8     else
9        $W_t := W_T$ 
10    end
11     $\text{updateDataPoints}(W_{t-k}, W_t)$ 
12    for  $i = (t - k) \rightarrow (t - 1)$  do
13       $\mathbb{H}.\text{append}(h(D[\mathbb{I}[i]]))$ 
14    end
15  else
16     $\mathbb{W}.\text{append}(\text{nil})$ 
17  end
18 end
19
20 function  $\text{updateDataPoints}(W_{t-k}, W_t)$ 
21    $W'_{t-k} := W_{t-k}$ 
22   for  $i = (t - k) \rightarrow (t - 1)$  do
23      $(\mathbf{X}, \mathbf{y}) \leftarrow D[\mathbb{I}[i]]$ 
24      $W'_{i+1} \leftarrow \text{update}(W'_i, (\mathbf{X}, \mathbf{y}))$ 
25     while  $d(W'_{i+1}, W'_i) > \gamma$  do
26        $\mathbf{R} \leftarrow \text{zeros}$ 
27        $\nabla_{W'_i} \leftarrow -\frac{\partial}{\partial W'_i} L(f_{W'_i}(\mathbf{X} + \mathbf{R}), \mathbf{y})$ 
28        $\mathbb{D}_i \leftarrow d(\nabla_{W'_i}, 0) + d(\mathbf{R}, 0)$ 
29        $\mathbf{R} \leftarrow \mathbf{R} - \eta \nabla_{W'_i} \mathbb{D}_i$ 
30        $W'_{i+1} \leftarrow \text{update}(W'_i, (\mathbf{X} + \mathbf{R}, \mathbf{y}))$ 
31     end
32      $D[\mathbb{I}[i]] := (\mathbf{X} + \mathbf{R}, \mathbf{y})$ 
33   end
34 end
```

Algorithm 4 shows Attack II. We highlight the key differences (compared to Attack I) in blue.

This time, $\mathcal{A}$ initializes W_0 via $\text{init}W_0$ (line 2), which ensures that W_0 follows the given distribution ζ , and minimizes $d(W_0, W_T)$ at the same time. It works as follows:

- 1) Suppose there are n elements in W_T , $\mathcal{A}$ puts these elements into a set S_1 . Then, $\mathcal{A}$ samples n elements: $v_1, \dots, v_n$ from the given distribution ζ , and puts them into another set V_2 .
- 2) $\mathcal{A}$ finds the largest elements w and v from S_1 and S_2 respectively. Then, $\mathcal{A}$ puts v into W_0 according to w 's indices in W_T .
- 3) $\mathcal{A}$ removes (w, v) from (S_1, S_2) , and repeats step 2) until S_1 and S_2 are empty.

Our experimental results show that this process can initialize a W_0 that meets our requirements.

For other W_t s ($t > 0$), $\mathcal{A}$ can initialize them by equally dividing the distance between W_0 and W_T . If the number of steps for spoofing (i.e., T') is large enough (i.e., there are enough W_t s), the condition “ $d(W_t, W_{t-k}) \leq \delta$ ” can be trivially satisfied.

Another major change in Attack II is that $\mathcal{A}$ optimizes the noise $\mathbf{R}$ by minimizing the following distance (line 27):

$$\mathbb{D}_i \leftarrow d(\nabla_{W'_i}, 0) + d(\mathbf{R}, 0),$$

and the condition for terminating the adversarial optimization is $d(W'_{i+1}, W'_i) > \gamma$ where $\gamma \ll \delta$ (Line 25). This guarantees that the model is still close to itself after a single step of update. Since the distance between W_{t-k} and W_t is smaller than δ after initialization, after k steps of updates, their distance is still smaller than δ : $d(W_t, W_{t-k}) < \delta$.

Interestingly, this change makes the adversarial optimization become easier to converge. Recall that in Attack I, $\mathcal{A}$ has to adjust the loss function $L(f_{W'_i}(\mathbf{X} + \mathbf{R}), \mathbf{y})$ to minimize $d(W'_{t-1} + \eta \nabla_{W'_{t-1}}, W_t)$. This is difficult to achieve because gradient-based training is used to minimize (not adjust) the loss function. Thanks to the new $\mathbb{D}_i$, $\mathcal{A}$ can simply minimize the loss function in Attack II. In another word, the adversarial optimization process in Attack II is more close to normal training. Table II shows that on CIFAR-10, after 10 steps of adversarial optimization, the loss function decreases from 0.43 to 0.04, and the gradients decrease from 61.13 to 0.12. Both are small enough to pass the verification. That is to say, the number of while loops N can be as small as 10 in Attack II.

Table II
THE CHANGES OF LOSS AND THE GRADIENTS AFTER 20 STEPS OF
ADVERSARIAL OPTIMIZATION ON CIFAR-10

	$L(f_{W'_i}(\mathbf{X} + \mathbf{R}), \mathbf{y})$	$\ \nabla_{W'_i}\ ^2$
Before	0.43 ± 0.18	61.13 ± 45.86
After	0.04 ± 0.01	0.12 ± 0.05

Cost comparison. It is easy to see that Attack II requires T' times of update (Line 24) plus $T' \cdot N$ times of adversarial optimization (Line 26-30), where $N = 10$. Each update requires one gradient computation and each adver-

serial optimization requires three gradient computations. In total, Attack II requires $31T'$ gradient computations. As a comparison, generating a PoL proof requires $T = E \cdot S$ gradient computations. In [12], they set $E = 200$ and $S = 390$, hence $T = 78,000$. Therefore, the cost for Attack II is smaller than the trainer's cost as long as $T' < \frac{78,000}{31} \approx 2,516$. Our experimental results show that this is more than enough for the spoof to pass the verification (cf. Section IV). Next, we show how we further optimize our attack.

C. Attack III

Algorithm 5: Attack III

Input: $D, f_{W_T}, \delta, \gamma, \zeta, k, E, S$
Output: PoL spoof: $\mathcal{P}(\mathcal{A}, f_{W_T}) = (\mathbb{W}, \mathbb{I}, \mathbb{H})$
 updated dataset: D

```

1  $\mathbb{W} \leftarrow \{\}$   $\mathbb{I} \leftarrow \{\}$   $\mathbb{H} \leftarrow \{\}$ 
2  $\mathbb{W}.\text{append}(\text{initW}_0(\zeta, W_T))$            initialize and
    $\text{append } W_0$ 
3 for  $t = 1 \rightarrow T'$  do                        $T' \bmod k = 0$ 
4    $\mathbb{I}.\text{append}(\text{getBatch}(D))$ 
5   if  $t \bmod k = 0$  then
6     if  $t < T$  then           no need to append  $W_T$ 
7       sample  $W_t$  s.t.,  $d(W_t, W_{t-k}) \leq \delta$ 
        $\mathbb{W}.\text{append}(W_t)$ 
8     else
9        $W_t := W_T$ 
10    end
11     $\text{updateDataPoints}(W_{t-k}, W_t)$ 
12    for  $i = (t-k) \rightarrow (t-1)$  do
13       $\mathbb{H}.\text{append}(h(D[\mathbb{I}[i]]))$ 
14    end
15  else
16     $\mathbb{W}.\text{append}(\text{nil})$ 
17  end
18 end
19
20 function  $\text{updateDataPoints}(W_{t-k}, W_t)$ 
21    $(\mathbf{X}, \mathbf{y}) \leftarrow [D[\mathbb{I}[t-k]] \dots D[\mathbb{I}[t-1]]]$ 
22    $W'_t \leftarrow \text{update}(W_{t-k}, (\mathbf{X}, \mathbf{y}))$ 
23   while  $d(W'_t, W_t) > \gamma - \sigma$  do
24      $\mathbf{R} \leftarrow \text{zeros}$ 
25      $\nabla_{W_{t-k}} \leftarrow -\frac{\partial}{\partial W_{t-k}} L(f_{W_{t-k}}(\mathbf{X} + \mathbf{R}), \mathbf{y})$ 
26      $\mathbb{D}_{t-k} \leftarrow d(\nabla_{W_{t-k}}, 0) + d(\mathbf{R}, 0)$ 
27      $\mathbf{R} \leftarrow \mathbf{R} - \eta' \nabla_{\mathbf{R}} \mathbb{D}_{t-k}$ 
28      $W'_t \leftarrow \text{update}(W_{t-k}, (\mathbf{X} + \mathbf{R}, \mathbf{y}))$ 
29   end
30    $[D[\mathbb{I}[t-k]] \dots D[\mathbb{I}[t-1]]] := (\mathbf{X} + \mathbf{R}, \mathbf{y})$ 
31 end
```

Algorithm 5 shows Attack III. Again, we highlight the key differences (compared to Attack II) in blue. The major change is that $\mathcal{A}$ optimizes all k batches of data points together in updateDataPoints . The distance between W'_t and W_t should be limited to $\gamma - \sigma$ (where $0 < \sigma < \gamma$), instead of γ . We will show the reason in the proof of Corollary 1.

Optimizing all k batches together reduces the complexity to $\frac{T'}{k}$ times of update (Line 22) plus $\frac{T'}{k} \cdot N$ times of adversarial optimization (Line 24-28). At first glance, this will not pass the verification because $\mathcal{V}$ will run update for each batch individually. However, since $W_{t-k}, \dots, W_{t-1}$ are all

very close to each other, taking a gradient with respect to W_{t-k+j} is similar to taking a gradient with respect to W_{t-k} . Consequently, we can optimize all k batches directly with respect to W_{t-k} . The gap only depends on k , hence we can make a trade-off. Next, we formally prove this argument.

Corollary 1. *Let (W_{t-k}, W_t) be an input to updateDataPoints in Attack III. Let $\{\hat{W}_{t-k-1}, \dots, \hat{W}_t\}$ be the model weights computed by $\mathcal{V}$ based on W_{t-k} during PoL verification. Assuming the loss function $L(f_W(\mathbf{X}), \mathbf{y}) \in C^2(\Omega)$, where Ω is a closed, convex and connected subset in $\mathbb{R}^n$, and $\{\hat{W}_{t-k-1}, \dots, \hat{W}_t\} \in \Omega$. Then,*

$$\|\hat{W}_t - W_t\| \leq \eta^2 \alpha \beta \frac{(k-1)(k-2)}{2} + \gamma - \sigma,$$

where α and β are the upper bounds of first and second order derivative³ of $L(f_W(\mathbf{X}), \mathbf{y})$.

Proof. Let $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_k]^T$ be the k batches used to update W_{t-k} . Denote

$$L_i(W) = L(f_W(\mathbf{x}_i), \mathbf{y}_i) \in C^2(\Omega),$$

$$\nabla_i(W) = \frac{\partial}{\partial W} L_i \in C^1(\Omega),$$

$$\nabla'_i(W) = \frac{\partial^2}{\partial W^2} L_i \in C^0(\Omega).$$

Then, $\|\nabla_i(W)\| < \alpha$ and $\|\nabla'_i(W)\| < \beta$.

In Attack III, (Line 22 of Algorithm 5), W'_t is calculated as

$$W'_t = W_{t-k} - \frac{\eta''}{k} (\nabla_1(W_{t-k}) + \nabla_2(W_{t-k}) + \dots + \nabla_k(W_{t-k}))$$

Whereas, in PoL verification (Line 29 of Algorithm 2),

$$\hat{W}_{t-k+1} = W_{t-k} - \eta \nabla_1(W_{t-k})$$

$$\hat{W}_{t-k+2} = \hat{W}_{t-k+1} - \eta \nabla_2(\hat{W}_{t-k+1})$$

$$\dots$$

$$\hat{W}_t = \hat{W}_{t-1} - \eta \nabla_k(\hat{W}_{t-1})$$

It is identical to

$$\hat{W}_t = W_{t-k} - \eta (\nabla_1(W_{t-k}) + \nabla_2(\hat{W}_{t-k+1}) + \dots + \nabla_k(\hat{W}_{t-1}))$$

If $\mathcal{A}$ sets $\eta'' = k\eta$, then

$$\begin{aligned} \hat{W}_t - W'_t &= \eta [(\nabla_2(W_{t-k}) - \nabla_2(\hat{W}_{t-k+1}) + \\ &\quad (\nabla_3(W_{t-k}) - \nabla_3(\hat{W}_{t-k+2}) + \dots + \\ &\quad (\nabla_k(W_{t-k}) - \nabla_k(\hat{W}_{t-1})) \end{aligned}$$

Assuming $[\hat{W}_{t-k+l}, W_{t-k}] = \{W \in \mathbb{R}^n, W = \hat{W}_{t-k+l} + \theta h, 0 \leq \theta \leq 1\}$ is a closed set, and $\nabla_i(W) \in C^1(\Omega)$. Based on the finite-increment theorem [29], we have

$$\begin{aligned} \|\nabla_i(W_{t-k}) - \nabla_i(\hat{W}_{t-k+l})\| &\leq \sup_W \left\| \frac{\partial \nabla_i(W)}{\partial W} \right\| \cdot \|h\| \\ &\leq \beta \|W_{t-k} - \hat{W}_{t-k+l}\| \end{aligned}$$

³Empirically, α is 0.03 in average and β is 0.025 in average.

Given

$$\begin{aligned} \|W_{t-k} - \hat{W}_{t-k+l}\| &= \eta \|\nabla_1(W_{t-k}) + \nabla_2(\hat{W}_{t-k+1}) + \dots \\ &\quad + \nabla_{l-1}(\hat{W}_{t-k+l-1})\| \\ &\leq (l-1)\eta\alpha, \end{aligned}$$

we have

$$\|\nabla_i(W_{t-k}) - \nabla_i(\hat{W}_{t-k+l})\| \leq (l-1)\eta\alpha\beta.$$

Then,

$$\begin{aligned} \hat{W}_t - W'_t &\leq \eta^2\alpha\beta \sum_{l=1}^{k-1} (l-1) \\ &= \eta^2\alpha\beta \frac{(k-1)(k-2)}{2} \end{aligned}$$

Recall that $d(W'_t, W_t) \leq \gamma - \sigma$ (Line 23 in Algorithm 5). Then,

$$\begin{aligned} \|\hat{W}_t - W_t\| &= \|\hat{W}_t - W'_t + W'_t - W_t\| \\ &\leq \|\hat{W}_t - W'_t\| + \|W'_t - W_t\| \\ &\leq \eta^2\alpha\beta \frac{(k-1)(k-2)}{2} + \gamma - \sigma \end{aligned}$$

□

Therefore, Attack III can pass the verification if we set $\sigma > \eta^2\alpha\beta \frac{(k-1)(k-2)}{2}$.

Cost comparison. Recall that the complexity for Attack III is $\frac{T'}{k}$ times of update plus $\frac{T'}{k} \cdot N$ times of adversarial optimization, where $N = 10$; each update requires one gradient computation and each adversarial optimization requires three gradient computations. In total, Attack III requires $31\frac{T'}{k}$ gradient computations. Given that generating a PoL proof requires $T = 78,000$ gradient computations, the cost for Attack III is smaller than the trainer's cost as long as $\frac{T'}{k} < \frac{78,000}{31} \approx 2,516$. In our experiments, we show Attack III can pass the verification when we set $k = 100$ (cf. Section IV). Then, $T' < 251,600$ is the condition for the cost of Attack III to be smaller than the trainer's cost, making the parameter setting more flexible than Attack II. Notice that our experiments did not show such good results, because k batches of samples exceeds the memory size and we have to load them separately. Nevertheless, the results of Attack III are still much better than Attack II.

IV. EVALUATION

In this section, we evaluate our attacks in two metrics:

- **Reproducibility.** We need to show that the normalized reproduction errors (cf. Section II-D) in l_1 , l_2 , l_∞ and $\cos$ introduced by our PoL spoof are smaller than those introduced by a legitimate PoL proof. That means, as long as the PoL proof can pass the verification, our spoof can pass the verification as well.
- **Spoof cost.** Recall that a successful PoL spoof requires the attacker to spend less computation and storage than the prover (cf. Section II-A). Therefore, we need to show that the generation time and size of the spoof are smaller than those of the proof.

A. Setting

Following the experimental setup in [12], we evaluate our attacks for ResNet-20 [11] and ResNet-50 [11] on CIFAR-10 [14] and CIFAR-100 [14] respectively. Each of them contains 50,000 training images and 10,000 testing images; and each image is of size $32 \times 32 \times 3$. CIFAR-10 only has 10 classes and CIFAR-100 has 100 classes.

We reproduced the results in [12] as baselines for our attacks. Namely, we generate the PoL proof by training both models for 200 epochs with 390 steps in each epoch (i.e., $E = 200$, $S = 390$) with batch sizes being 128. The authors in [12] suggest to set the number of steps (k) in each checkpointing interval equal to S , i.e., $k = S$, but in that case the verifier can only verify one model per epoch, i.e., $Q = 1$ (cf. Algorithm 2). Therefore, we set k as 100. Under this setting, the models achieve 90.13% accuracy on CIFAR-10 and 77.03% accuracy on CIFAR-100. All reproduced results are consistent with the results reported in [12].

Since our attacks are stochastic spoofing, where the spoof is *not* required to be exactly the same as the proof, we could use different T , k and batch size in our attacks. Nevertheless, we still set k as 100 and batch size as 128 for all of our experiments, to show that our attacks are still valid even in a tough hyperparameter setting.

Recall that $\mathcal{V}$ only verifies $Q < \frac{S}{k}$ largest updates for each epoch (cf. Algorithm 2). Given that $S = 390$ and $k = 100$, we have $Q \leq 3$. In [12], Jia et al. claim that $Q = 1$ would be sufficient for the verifier to detect spoofing, and they use $Q = 1$ for their experiments. To be cautious, we run experiments with both $Q = \lfloor \frac{S}{k} \rfloor = 3$ and $Q = 1$.

To show that our attacks can scale to more complex datasets, we evaluate our attacks on ImageNet [8]. We pick the first 10 classes from ImageNet; it contains 13,000 training images and 500 testing images, and each image is resized as 128×128 . We train ResNet-18 [11] with $E = 200$, $S = 101$, $k = 100$ and batch size being 128; it achieves 71% accuracy. In this case, Q can only be one. We again set $k = 100$ and batch size as 128 for our attacks on ImageNet.

All experiments were repeated 5 times with different random initializations, and the averages are reported.

B. results

Attack I. Table III shows the evaluation results of Attack I on CIFAR-10. It shows that after a similar amount of training time, the normalized reproduction errors introduced by PoL spoof are significantly larger than those introduced by PoL proof. This is unsurprising since Attack I is just a strawman attack. Next, we focus on evaluating our real attacks: Attack II and Attack III.

Attack II. Figure 2 shows the evaluation results of Attack II on CIFAR-10. The results show that the normalized reproduction errors introduced by the PoL spoof are always smaller than those introduced by the PoL proof in l_1 , l_2 and $l_{\cos}$ (Figure 2(a), 2(b) and 2(d)). For l_∞ , it requires $T' > 1,000$ for the spoof to be able to pass the verification (Figure 2(c)). On

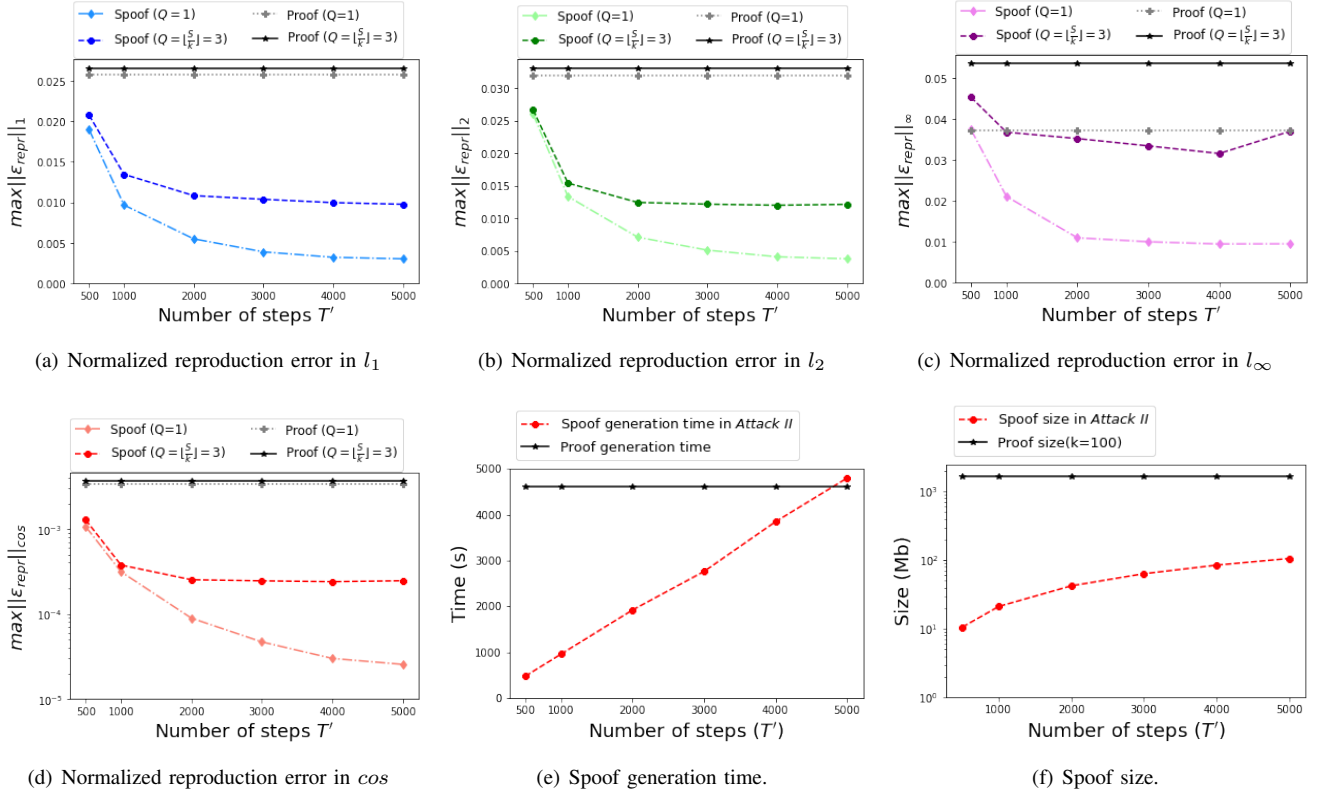


Figure 2. Attack II on CIFAR-10

Table III
ATTACK I ON CIFAR-10 AND CIFAR-100

	CIAFR-10		CIFAR-100	
	Spoof	Proof	Spoof	Proof
l_1	0.2152	0.0265	0.7558	0.1911
l_2	0.2796	0.0333	0.6552	0.3330
l_∞	0.2291	0.0283	0.1518	0.5075
$\cos$	0.0758	0.0038	0.1345	0.0043
Time (s)	4,591	4,607	18,307	17,756

the other hand, when $T' > 4,000$, the generation time of the spoof is larger than that of the proof (Figure 2(e)). That means $1,000 < T' < 4,000$ would be the condition for Attack II to succeed on CIFAR-10. Notice that the spoof size is always smaller than the proof size.

Figure 3 shows that $1,300 < T' < 1,700$ is the condition for Attack II to be successful on CIFAR-100. Figure 4 shows that $500 < T' < 3,000$ is the condition for Attack II to succeed on ImageNet.

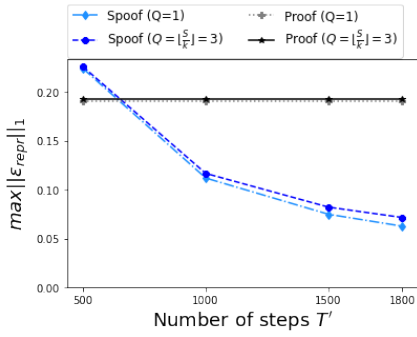
Attack III. Figure 5 shows the results of Attack III on CIFAR-10. The results show that the normalized reproduction errors introduced by the spoof are always smaller than those introduced by the proof in all 4 distances when $T' > 1,000$. In terms of spoof generation time, the number of steps T' can be as large as 6,000. That means the condition for Attack III

to be successful on CIFAR-10 is $1,000 < T' < 6,000$.

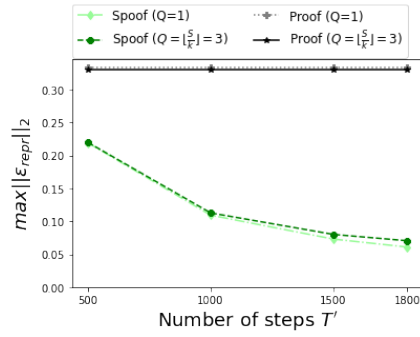
Figure 6 shows that the condition for Attack III to succeed in CIFAR-100 is $1,500 < T' < 4,000$. Figure 7 shows that $500 < T' < 18,000$ is the condition for Attack III to be successful on ImageNet.

In [12], the authors suggest to check model performance periodically. Therefore, we need to make sure that the performance of the intermediate models generated by our attacks follow the same trend as those in the proof. We can achieve this by adjusting the extent of perturbations on W_T (Line 7 in Algorithm 5). Specifically, we can add a large extent of perturbations when T' is small and add a small extent of perturbations when T' is large. Figure 10 (in Appendix) shows the model performance in both CIFAR-10 and CIFAR-100. The x -axis presents the progress of training. For example when $x = 0.2$, the corresponding y represents the $0.2 \cdot T'$ -th model performance in the proof and $0.2 \cdot T'$ -th model performance in the spoof. It shows that the model performance in the proof and the spoof are in similar trends.

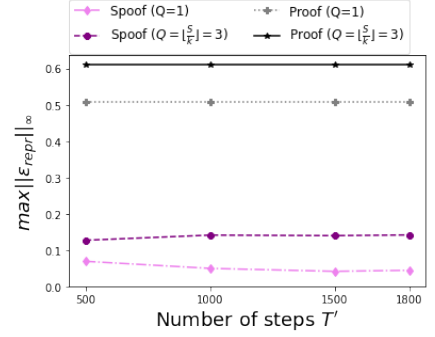
Non-overlapping datasets. In [12], it is assumed that $\mathcal{A}$ has full access to the training dataset and can modify it. An implicit assumption is that $\mathcal{V}$ does not know the dataset beforehand, otherwise the attack can be easily defended by checking the integrity of the dataset. This assumption is realistic. Consider the scenario where two hospitals share data with each other, so that they can train models (separately) for online diagnosis. Suppose hospital-A trains a good model, and



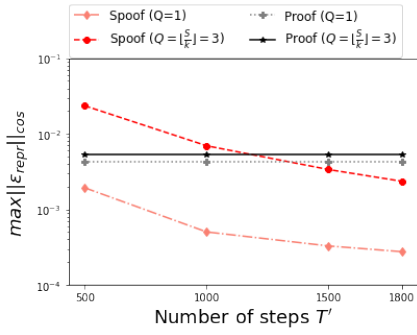
(a) Normalized reproduction error in l_1



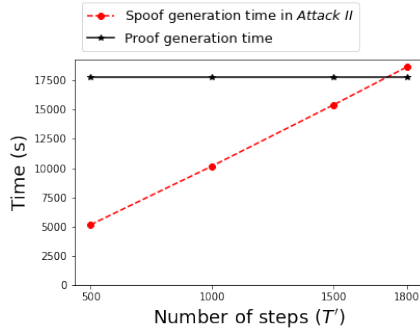
(b) Normalized reproduction error in l_2



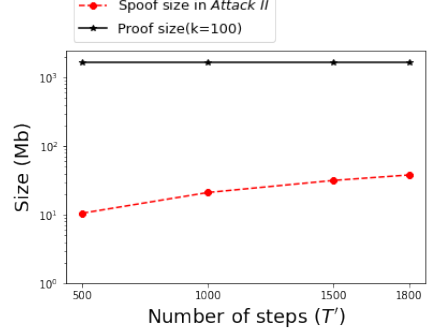
(c) Normalized reproduction error in l_∞



(d) Normalized reproduction error in $\cos$

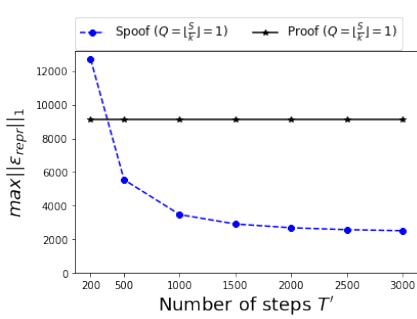


(e) Spoof generation time.

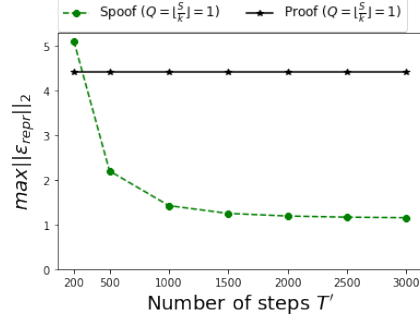


(f) Spoof size.

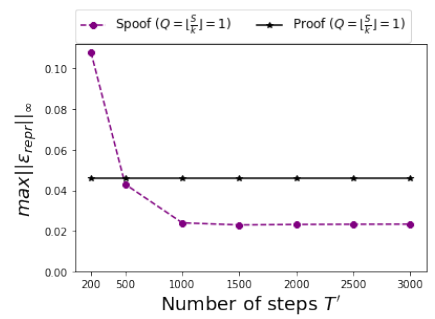
Figure 3. Attack II on CIFAR-100



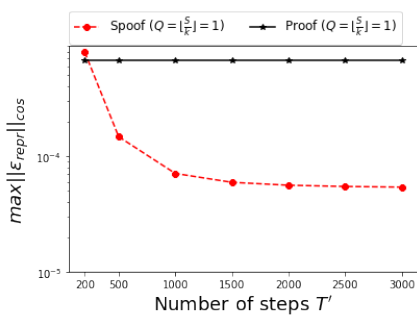
(a) Normalized reproduction error in l_1



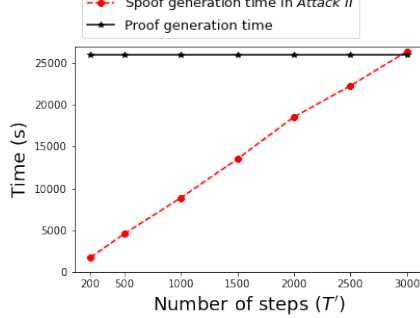
(b) Normalized reproduction error in l_2



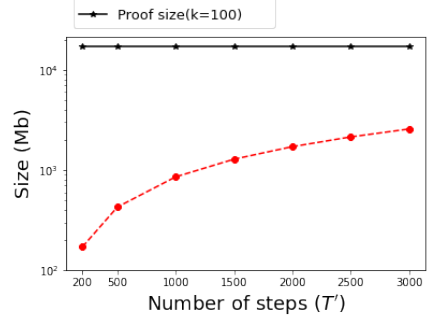
(c) Normalized reproduction error in l_∞



(d) Normalized reproduction error in $\cos$



(e) Spoof generation time.



(f) Spoof size.

Figure 4. Attack II on ImageNet

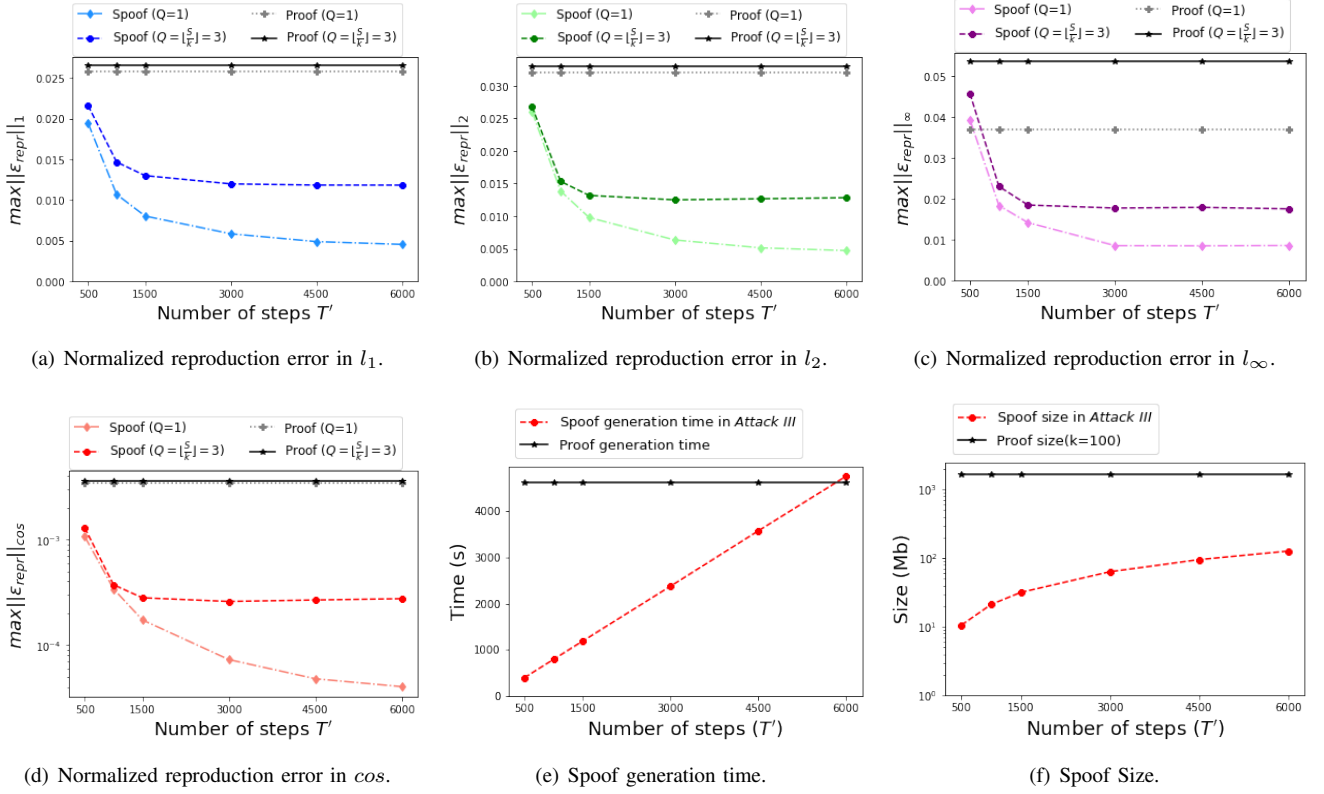


Figure 5. Attack III on CIFAR-10.

hospital-B (which is the attacker) wants to claim the ownership of this model. Then, hospital-A provides a PoL proof and hospital-B provides a PoL spoof. In this example, $\mathcal{A}$ knows the training dataset but $\mathcal{V}$ does not.

However, $\mathcal{V}$ could still check if one dataset is an adversarial perturbation of the other. This may introduce a new arms race: $\mathcal{A}$ would need to evade both PoL and this second verifier. To this end, we evaluate the effectiveness of our attacks on two non-overlapping datasets. Namely, we split the dataset CIFAR-10 into two non-overlapping datasets: D_1 and D_2 . $\mathcal{T}$ generates the PoL proof from D_1 and $\mathcal{A}$ generates the PoL spoof from D_2 . In this case, $\mathcal{V}$ can never know which dataset is an adversarial perturbation. Figure 8 and Figure 9 show that we can still spoof PoL when the two training datasets are non-overlapping. This is intuitively true; even for the same dataset, the `getBatch` function randomly samples data points, hence the data points used for spoofing are w.h.p. different from those used for generating PoL. That means our attack could always succeed with a different dataset.

Different hardware. Next, we show the spoof generation time is smaller than the proof generation time even if $\mathcal{A}$ uses a less powerful hardware than $\mathcal{T}$. We pick four kinds of hardware and rank them in ascending order according to their computing power: Tesla T4, Tesla P40, GeForce RTX 2080Ti and Tesla V100. We run Attack II and Attack III on all these hardware. In addition, we use the most powerful hardware (i.e., Tesla V100) to generate the PoL proof. Figure 11 (in

Appendix) shows that the attacks can succeed even if we use the least powerful hardware to generate spoof and use the most powerful hardware to generate proof.

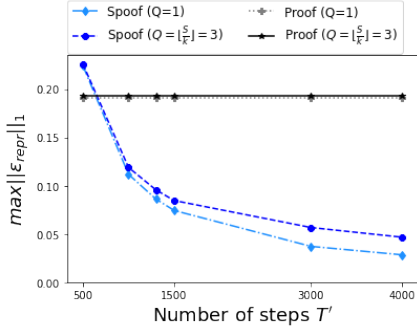
Summary. In summary, both Attack II and Attack III can successfully spoof a PoL when T' is within a certain range. The results show that T' can be large enough to be in the same level with T . Consequently, $\mathcal{V}$ cannot simply detect the attack by setting a lower bound on the number of training steps.

Figure 12 (in Appendix) shows some randomly picked images before and after running our attacks. The differences are invisible.

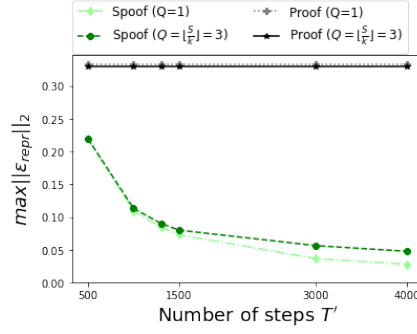
V. COUNTERMEASURES

In this section, we provide two potential countermeasures for our attacks.

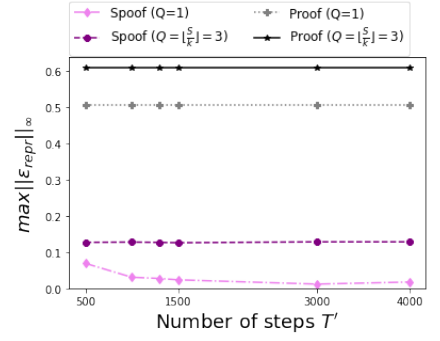
Selection of threshold. As we observed in our experiments, ϵ_{repr} in the early training stage is usually larger than that in the later stage, because the model converges in the later stage. On the other hand, ϵ_{repr} remains in the same level in our spoof. Then, it is unreasonable for $\mathcal{V}$ to set a single verification threshold for the whole training stage. A more sophisticated way would be dynamically choosing the verification thresholds according to the stage of model training: choose larger thresholds for the early stage, and choose smaller thresholds for the later stage. However, we can also set $d(W_t, W_{t-k})$ closer in the later stage to circumvent this countermeasure.



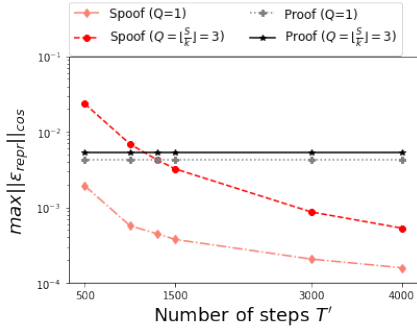
(a) Normalized reproduction error in l_1 .



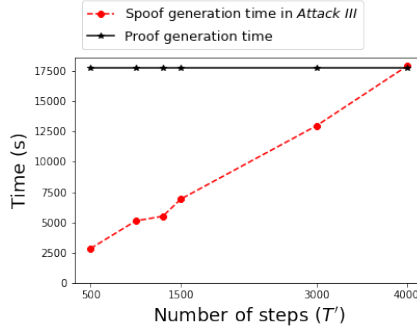
(b) Normalized reproduction error in l_2 .



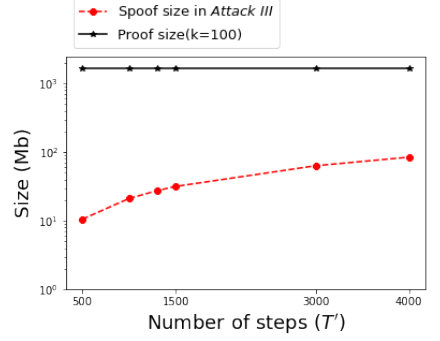
(c) Normalized reproduction error in l_∞ .



(d) Normalized reproduction error in $\cos$.

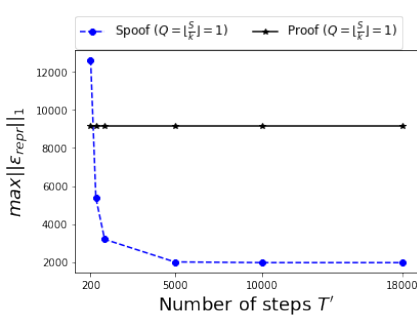


(e) Spoof generation time.

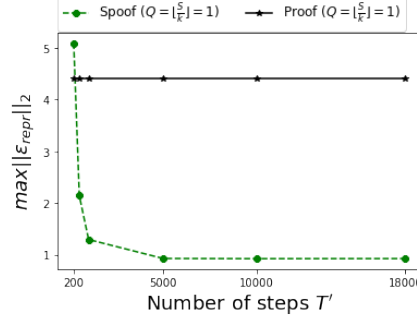


(f) Spoof Size.

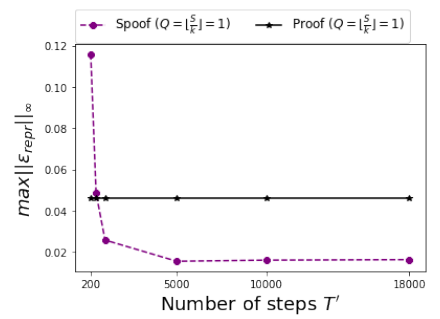
Figure 6. Attack III on CIFAR-100.



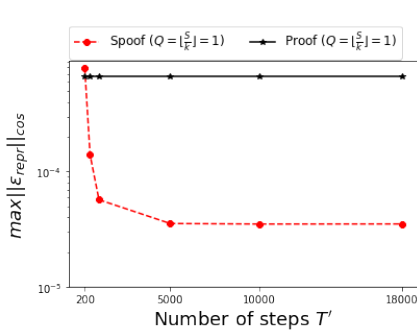
(a) Normalized reproduction error in l_1 .



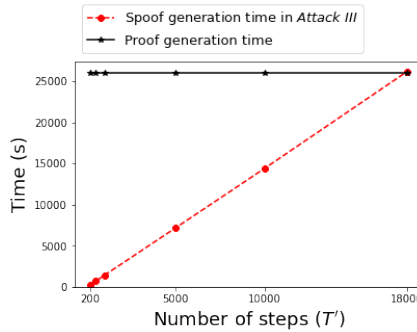
(b) Normalized reproduction error in l_2 .



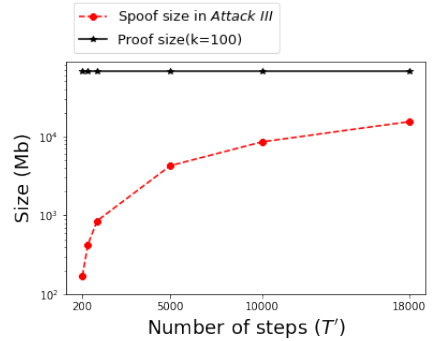
(c) Normalized reproduction error in l_∞ .



(d) Normalized reproduction error in $\cos$.

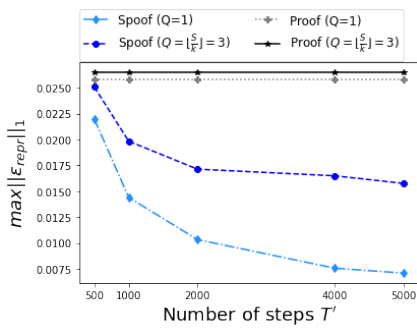


(e) Spoof generation time.

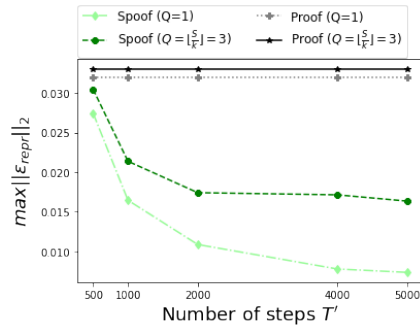


(f) Spoof Size.

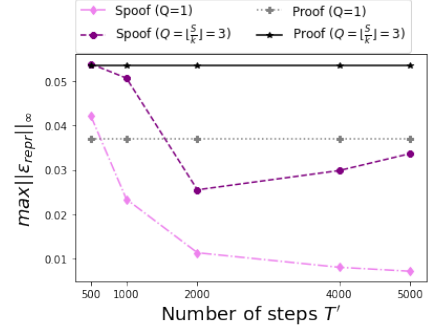
Figure 7. Attack III on ImageNet.



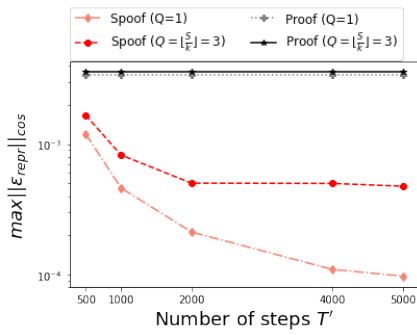
(a) Normalized reproduction error in l_1



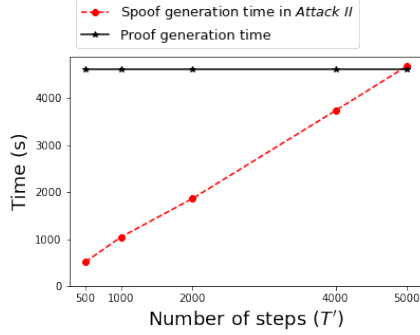
(b) Normalized reproduction error in l_2



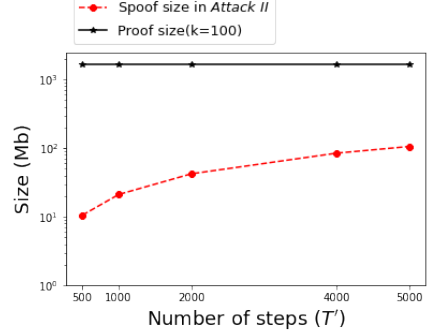
(c) Normalized reproduction error in l_∞



(d) Normalized reproduction error in $\cos$

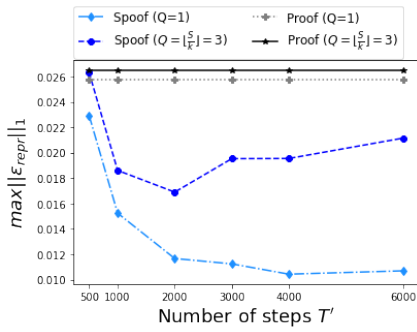


(e) Spoof generation time.

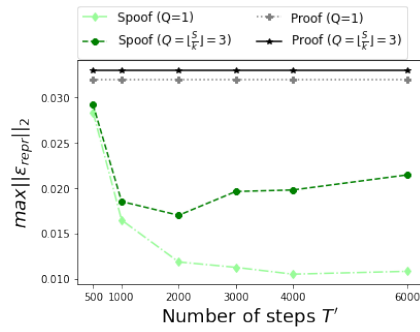


(f) Spoof size.

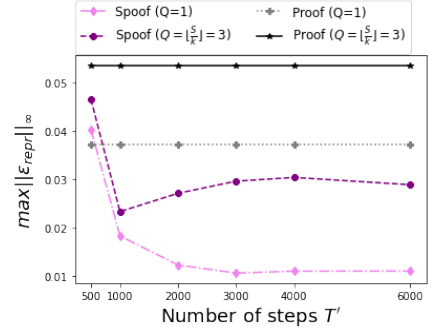
Figure 8. Attack II on non-overlapping CIFAR-10



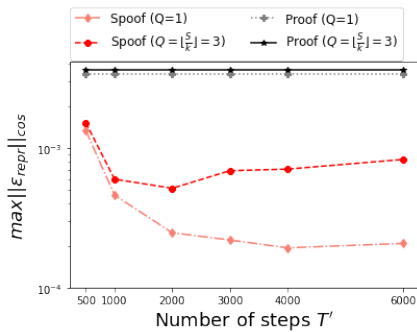
(a) Normalized reproduction error in l_1 .



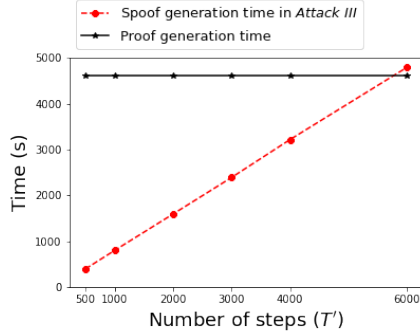
(b) Normalized reproduction error in l_2 .



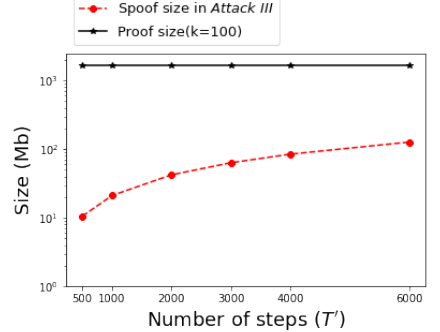
(c) Normalized reproduction error in l_∞ .



(d) Normalized reproduction error in $\cos$.



(e) Spoof generation time.



(f) Spoof Size.

Figure 9. Attack III on non-overlapping CIFAR-10.

VC-based PoL. Verifiable computation (VC) allows a delegator to outsource the execution of a complex function to some workers, which return the execution result together with a VC-proof; the delegator can check the correctness of the returned results by verifying the VC-proof, which requires less work than executing the function. There are many VC schemes such as SNARK [2], [20], STARK [1] etc; most of them require $O(n \log n)$ computational complexity to generate the proof, where n is the number of gates in the function. We can use VC to build a secure PoL mechanism: during model training, $\mathcal{T}$ generates a VC-proof, proving that the final model W_T was resulted by running the training algorithm on the initial model W_0 and the dataset D ; when the model ownership is under debate, $\mathcal{T}$ can show the VC-proof to $\mathcal{V}$. To spoof, $\mathcal{A}$ has to devote $O(n \log n)$ computation to generate a valid VC-proof, which is almost equal as $\mathcal{T}$. This mechanism is valid, but it will introduce an overwhelming overhead.

VI. RELATED WORK

Adversarial examples. When first discovered in 2013 [23], adversarial examples are images designed intentionally to cause deep neural networks to make false predictions. Such adversarial examples look almost the same as original images, thus they show vulnerabilities of deep neural networks [10]. Since then, it becomes a popular research topic and has been explored extensively in both attacks [9], [22] and defences [3], [6], [16], [19], [26], [27].

Adversarial examples are generated by solving:

$$\mathbf{R} = \arg \min_{\mathbf{R}} L(f_W(\mathbf{X} + \mathbf{R}), \mathbf{y}') + \alpha \|\mathbf{R}\|,$$

where $\mathbf{y}'$ is a label that is different from the real label $\mathbf{y}$ for $\mathbf{X}$. Then, the noise $\mathbf{R}$ can fool the model to predict a wrong label (by minimizing the loss function) and pose little influence on the original instance $\mathbf{X}$.

Recall that the objective of Attack II and Attack III is to minimize the gradients computed by “adversarial examples”. Therefore, it can be formulated as:

$$\mathbf{R} = \arg \min_{\mathbf{R}} L(f_W(\mathbf{X} + \mathbf{R}), \mathbf{y}) + \alpha \|\mathbf{R}\|.$$

This is identical to the objective of finding an adversarial example. An adversarial example aims to fool the model whereas our attacks aim to update a model to itself. Nevertheless, they end up at the same point, which explains the effectiveness of Attack II and Attack III.

Unadversarial examples. Unadversarial examples [21] target the scenarios where a system designer not only trains the model for predictions, but also controls the inputs to be fed into that model. For example, a drone operator who trains a landing pad detector can also modify the surface of the landing pads. Unadversarial examples are generated by solving:

$$\mathbf{R} = \arg \min_{\mathbf{R}} L(f_W(\mathbf{X} + \mathbf{R}), \mathbf{y}),$$

so that the new input $(\mathbf{X} + \mathbf{R})$ can be recognized better by the model f_W .

This is similar to the objective function of our attacks besides the regularization term we use to minimize the perturbations. The authors in [21] demonstrate the effectiveness of unadversarial examples via plenty of experimental results, which can also explain the success of our attacks.

Deep Leakage from Gradient. In federated learning [5], [13], [15], [17]), it was widely believed that shared gradients will not leak information about the training data. However, Zhu et al. [28] proposed “Deep Leakage from Gradients” (DLG), where the training data can be recovered through gradients matching. Specifically, after receiving gradients from another worker, the adversary feeds a pair of randomly initialized dummy instance (X, y) into the model, and obtains the dummy gradients via back-propagation. Then, they update the dummy instance with an objective of minimizing the distance between the dummy gradients and the received gradients:

$$\begin{aligned} \mathbf{X}, y &= \arg \min_{\mathbf{X}, y} \|\nabla W' - \nabla W\|^2 \\ &= \arg \min_{\mathbf{X}, y} \left\| \frac{\partial L(f_W(\mathbf{X}), y)}{\partial W} - \nabla W \right\|^2 \end{aligned}$$

After a certain number of steps, the dummy instance can be recovered to the original training data.

Our attacks were largely inspired by DLG: an instance can be updated so that its output gradients can match the given gradients. The difference between DLG and our work is that DLG aims to recover the training data from the gradients, whereas we want to create a perturbation on a real instance to generate specific gradients.

VII. CONCLUSION

In this paper, we show that a recently proposed PoL mechanism is vulnerable to “adversarial examples”. Namely, in a similar way as generating adversarial examples, we could generate a PoL spoof with significantly less cost than generating a proof by the prover. We validated our attacks by conducting experiments extensively. In future work, we will explore more effective attacks.

ACKNOWLEDGMENTS

The work was supported in part by National Key Research and Development Program of China under Grant 2020AAA0107705, National Natural Science Foundation of China (Grant No. 62002319, 11771393, U20A20222) and Zhejiang Key R&D Plans (Grant No. 2021C01116).

REFERENCES

- [1] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. *IACR Cryptol. ePrint Arch.*, 2018:46, 2018.
- [2] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. Succinct non-interactive zero knowledge for a von neumann architecture. In *23rd {USENIX} Security Symposium ({USENIX} Security 14)*, pages 781–796, 2014.
- [3] Arjun Nitin Bhagoji, Daniel Cullina, Chawin Sitawarin, and Prateek Mittal. Enhancing robustness of machine learning systems via data transformations. In *2018 52nd Annual Conference on Information Sciences and Systems (CISS)*, pages 1–5, 2018.

- [4] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. Machine learning with adversaries: Byzantine tolerant gradient descent. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
- [5] Keith Bonawitz, Hubert Eichner, Wolfgang Grieskamp, Dzmitry Huba, Alex Ingerman, Vladimir Ivanov, Chloe Kiddon, Jakub Konečný, Stefano Mazzocchi, H Brendan McMahan, et al. Towards federated learning at scale: System design. *arXiv preprint arXiv:1902.01046*, 2019.
- [6] Jacob Buckman, Aurko Roy, Colin Raffel, and Ian Goodfellow. Thermometer encoding: One hot way to resist adversarial examples. In *International Conference on Learning Representations*, 2018.
- [7] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. cudnn: Efficient primitives for deep learning, 2014.
- [8] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009.
- [9] Yinpeng Dong, Fangzhou Liao, Tianyu Pang, Hang Su, Jun Zhu, Xiaolin Hu, and Jianguo Li. Boosting adversarial attacks with momentum. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 9185–9193, 2018.
- [10] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*, 2014.
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*, pages 770–778. IEEE Computer Society, 2016.
- [12] Hengrui Jia, Mohammad Yaghini, Christopher A. Choquette-Choo, Natalie Dullerud, Anvith Thudi, Varun Chandrasekaran, and Nicolas Papernot. Proof-of-learning: Definitions and practice. In *IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 2021.
- [13] Peter Kairouz, H Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. Advances and open problems in federated learning. *arXiv preprint arXiv:1912.04977*, 2019.
- [14] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.
- [15] Tian Li, Anit Kumar Sahu, Ameet Talwalkar, and Virginia Smith. Federated learning: Challenges, methods, and future directions. *IEEE Signal Processing Magazine*, 37(3):50–60, 2020.
- [16] Yan Luo, Xavier Boix, Gemma Roig, Tomaso Poggio, and Qi Zhao. Foveation-based mechanisms alleviate adversarial examples, 2016.
- [17] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In Aarti Singh and Jerry Zhu, editors, *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, volume 54, pages 1273–1282, Fort Lauderdale, FL, USA, 20–22 Apr 2017. PMLR.
- [18] Tribhuvanesh Orekondy, Bernt Schiele, and Mario Fritz. Knockoff nets: Stealing functionality of black-box models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4954–4963, 2019.
- [19] Nicolas Papernot, Patrick McDaniel, Xi Wu, Somesh Jha, and Ananthram Swami. Distillation as a defense to adversarial perturbations against deep neural networks. In *2016 IEEE symposium on security and privacy (SP)*, pages 582–597. IEEE, 2016.
- [20] B. Parno, J. Howell, C. Gentry, and M. Raykova. Pinocchio: Nearly practical verifiable computation. In *2013 IEEE Symposium on Security and Privacy*, pages 238–252, May 2013.
- [21] Hadi Salman, Andrew Ilyas, Logan Engstrom, Sai Vemprala, Aleksander Madry, and Ashish Kapoor. Unadversarial examples: Designing objects for robust vision. *arXiv preprint arXiv:2012.12235*, 2020.
- [22] Jiawei Su, Danilo Vasconcellos Vargas, and Kouichi Sakurai. One pixel attack for fooling deep neural networks. *IEEE Transactions on Evolutionary Computation*, 23(5):828–841, 2019.
- [23] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. *arXiv preprint arXiv:1312.6199*, 2013.
- [24] Florian Tramèr, Fan Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. Stealing machine learning models via prediction apis. In *25th {USENIX} Security Symposium ({USENIX} Security 16)*, pages 601–618, 2016.
- [25] Binghui Wang and Neil Zhenqiang Gong. Stealing hyperparameters in machine learning. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 36–52. IEEE, 2018.
- [26] Qinglong Wang, Wenbo Guo, Kaixuan Zhang, Alexander G. Ororbia II au2, Xinyu Xing, Xue Liu, and C. Lee Giles. Learning adversary-resistant deep neural networks, 2017.
- [27] Stephan Zheng, Yang Song, Thomas Leung, and Ian Goodfellow. Improving the robustness of deep neural networks via stability training. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4480–4488, 2016.
- [28] Ligeng Zhu and Song Han. Deep leakage from gradients. In *Federated learning*, pages 17–31. Springer, 2020.
- [29] Vladimir A. Zorich. *Mathematical Analysis II*. Universitext. Springer, 2nd edition, 2016.

APPENDIX

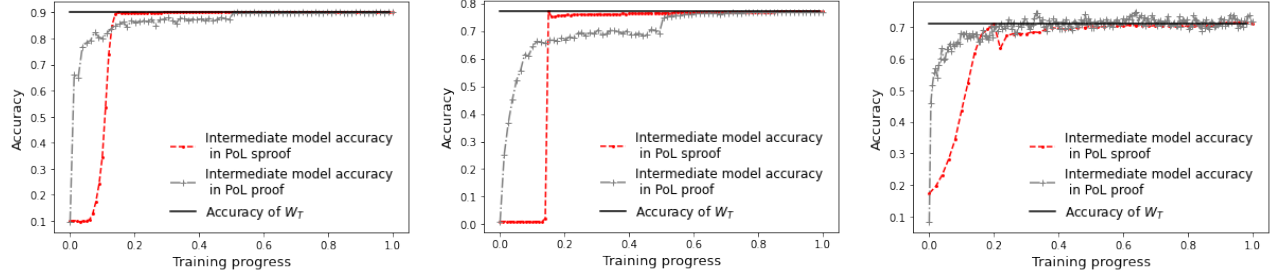


Figure 10. Intermediate model accuracy for Attack III. (The x -axis presents the progress of training. For example when $x = 0.2$, the corresponding y represents the $0.2 \cdot T$ -th model performance in PoL proof and $0.2 \cdot T'$ -th model performance in PoL spoof)

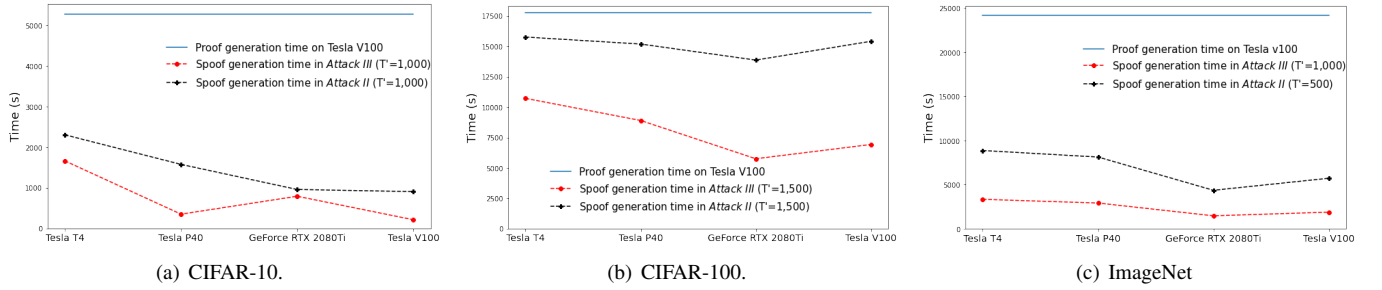


Figure 11. Spoof generation time with different GPUs. Tesla T4 is the least powerful one and Tesla V100 is the most powerful one. We use Tesla V100 to generate the proof.

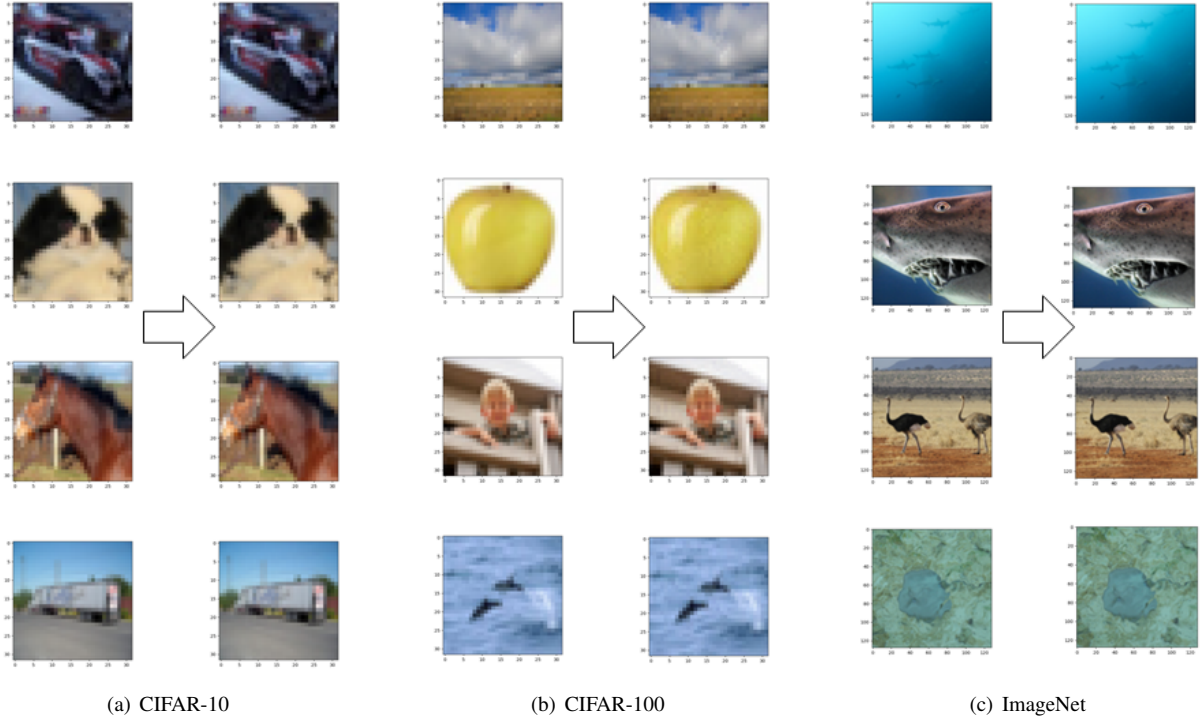


Figure 12. “Adversarial examples” generated by Attack III. The original images are on the left-hand side and the noised images are on the right-hand side.